

Regresión simbólica para la expresión analítica de soluciones de ecuaciones no lineales

Noel Pérez Calvo

Febrero 2026

OBJETIVO

Desarrollar y validar una metodología basada en regresión simbólica para determinar las expresiones analíticas de soluciones de ecuaciones no lineales.

QUÉ SE HA HECHO HASTA HOY

- Revisión del estado del arte
 - Se revisaron 30 artículos sobre regresión simbólica, búsqueda de raíces y álgebra computacional
 - Se estudiaron las bibliotecas de regresión simbólica existentes (PySR, gplearn, entre otras)
 - Se seleccionó PySR como herramienta base por su rendimiento en benchmarks y su backend en Julia
- Diseño del sistema: pipeline de 6 etapas
 - La idea general es: dada una ecuación $F(x; \alpha_1, \dots, \alpha_n) = 0$, resolver numéricamente para muchas combinaciones de parámetros y luego usar regresión simbólica para descubrir la fórmula $x = g(\alpha_1, \dots, \alpha_n)$
 - Se empezó por ecuaciones de una variable como caso de validación
- Etapa 1 — Definición de la ecuación
 - Parser simbólico con SymPy que acepta cualquier ecuación como texto
 - Detecta automáticamente cuáles símbolos son variables y cuáles son parámetros
- Etapa 2 — Generación del grid de parámetros
 - Genera todas las combinaciones de valores de los parámetros mediante producto cartesiano
 - Los rangos y la cantidad de puntos por parámetro son configurables
- Etapa 3 — Resolución numérica de los ceros

- Para cada combinación de parámetros, resuelve la ecuación con SymPy (`solve` y `nsolve`)
- Filtra raíces complejas y maneja timeouts
- Etapa 4 — Agrupación de raíces por rama
 - Ordena las raíces reales y las agrupa en ramas (primera raíz, segunda raíz, etc.)
 - Genera un dataset de regresión independiente por cada rama
- Etapa 5 — Regresión simbólica (módulo central)
 - Se usa PySR, que implementa programación genética multi-población sobre Julia
 - Se diseñó una arquitectura de tres niveles encima de PySR:
 - Proceso iterativo: encuentra una expresión, quita los puntos que cubre, repite con los restantes
 - Múltiples intentos por iteración: ejecuta PySR 3 veces y se queda con el mejor resultado para compensar la aleatoriedad
 - Evaluación del Hall of Fame completo: revisa todas las ecuaciones candidatas, no solo la que PySR considera “mejor”
 - Decisiones de diseño importantes:
 - Se creó un operador `safe_sqrt(x) =  $\sqrt{|x|}$` para evitar NaN durante la evolución genética
 - ◊ Sin esto, PySR no puede descubrir fórmulas con raíces cuadradas
 - ◊ Se probó el mecanismo `bumper` de PySR y no funcionó porque mataba las expresiones con `sqrt` antes de que pudieran componerse
 - Se implementó y descartó una función de pérdida sigmoide
 - ◊ Fragmentaba la búsqueda: PySR encontraba fórmulas locales en vez de la global
 - ◊ Se usa MSE estándar, que funciona bien porque los datos no tienen ruido
 - Se usa batching (mini-batches de 200 puntos) para controlar el consumo de RAM
 - Se configuró alta exploración: $\sim 275,000$ ciclos evolutivos, 30 poblaciones, sin penalización por complejidad
- Etapa 6 — Construcción de la expresión final
 - Si el proceso iterativo encuentra varias funciones para una rama, las ensambla como expresión Piecewise
- Sistema de benchmark: 43 casos de prueba
 - 10 lineales, 14 cuadráticas, 6 cúbicas, 4 cuárticas, 1 quíntica, 8 especiales
 - Cada caso tiene la ecuación, las raíces esperadas y métricas automáticas
- Resultados del benchmark

- 43/43 casos ejecutados sin errores (100 % éxito operativo)
- 77.6 % de raíces descubiertas, 28/43 casos perfectos
- Todos los lineales perfectos
- Patrón de fallo: descubre $\sqrt{a}$ pero no $-\sqrt{a}$ (y similares con el signo negativo)
- Causa identificada: convergencia prematura de la búsqueda evolutiva por excesiva presión selectiva en el tournament selection de PySR

RECOMENDACIONES Y TRABAJO FUTURO

- Resolver el problema de las raíces negativas no descubiertas

Cuadro 1: Literatura relevante para la configuración evolutiva de PySR

Referencia	Aporte relevante
Goldberg & Deb (1991)	Demostraron matemáticamente que la presión selectiva controla la velocidad de convergencia. Mayor presión implica convergencia más rápida y pérdida de diversidad más rápida. Formalizaron la teoría del tournament selection.
Blickle & Thiele (1996)	Formalizaron que la intensidad de selección crece con el tamaño del torneo n . Con $n = 15$ la selección es mucho más agresiva que con $n = 8$.
Koza (1992)	Libro fundacional de la Programación Genética. Analizó cómo la probabilidad de crossover afecta el rendimiento: valores moderados ($\sim 5-10$ %) dan buen balance entre exploración y preservación.

- Iterar sobre la configuración de PySR hasta que el benchmark dé resultados satisfactorios
- Implementar un módulo de verificación algebraica con bases de Gröbner
 - Para comprobar formalmente que las expresiones descubiertas son soluciones de la ecuación
 - Solo aplicable a ecuaciones polinómicas
- Extender el sistema a sistemas de ecuaciones no lineales con múltiples incógnitas
- Agregar casos de prueba con ecuaciones trascendentes (trigonométricas, exponenciales)
- Pensar cómo verificar ecuaciones no polinómicas (donde Gröbner no aplica)